Customer Churn Intelligence System

Customer churn significantly impacts company revenue.

PROBLEM STATEMENT :

- Customer churn leads to significant revenue loss in the telecom industry.
- Retaining existing customers is more cost-effective than acquiring new ones.
- Businesses need a way to identify customers who are likely to churn.

The objective of this project is to:

- Predict customer churn using Machine Learning
- Identify key churn drivers
- Estimate revenue at risk
- Recommend retention strategies
- Calculate ROI of intervention strategies

This project combines Data Analytics + Machine Learning + Business Intelligence.

```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.3)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.model_selection import train_test_split, cross
from sklearn.preprocessing import LabelEncoder, StandardSca
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, roc_auc_score, roc_curve
)

try:
    from xgboost import XGBClassifier
    xgb_available = True
except:
    xgb_available = False

sns.set(style="whitegrid")
```

DATA LOADING

```
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineS
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

5 rows × 21 columns

```
url = "https://raw.githubusercontent.com/sinharitamvaraa/customer-churn-intelligence-system/refs/heads/main/WA_Fn-UseC_
df = pd.read_csv(url)
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSe
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

5 rows × 21 columns

DATA OVERVIEW

```
df.info()
df.describe()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

	0
customerID	0
Dependents	0
tenure	0

DATA INSPECTION

```
print("Shape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())
print("\nData Types:\n", df.dtypes)
```

```
Shape: (7043, 21)
```

```
Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Data Types:
customerID          object
gender              object
SeniorCitizen        int64
Partner             object
Dependents          object
tenure               int64
PhoneService        object
MultipleLines       object
InternetService     object
OnlineSecurity      object
OnlineBackup        object
DeviceProtection    object
TechSupport         object
StreamingTV         object
StreamingMovies     object
Contract            object
PaperlessBilling    object
PaymentMethod       object
MonthlyCharges       float64
TotalCharges         float64
Churn               object
dtype: int64
```

```
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
dtype: object
```

CLEANING

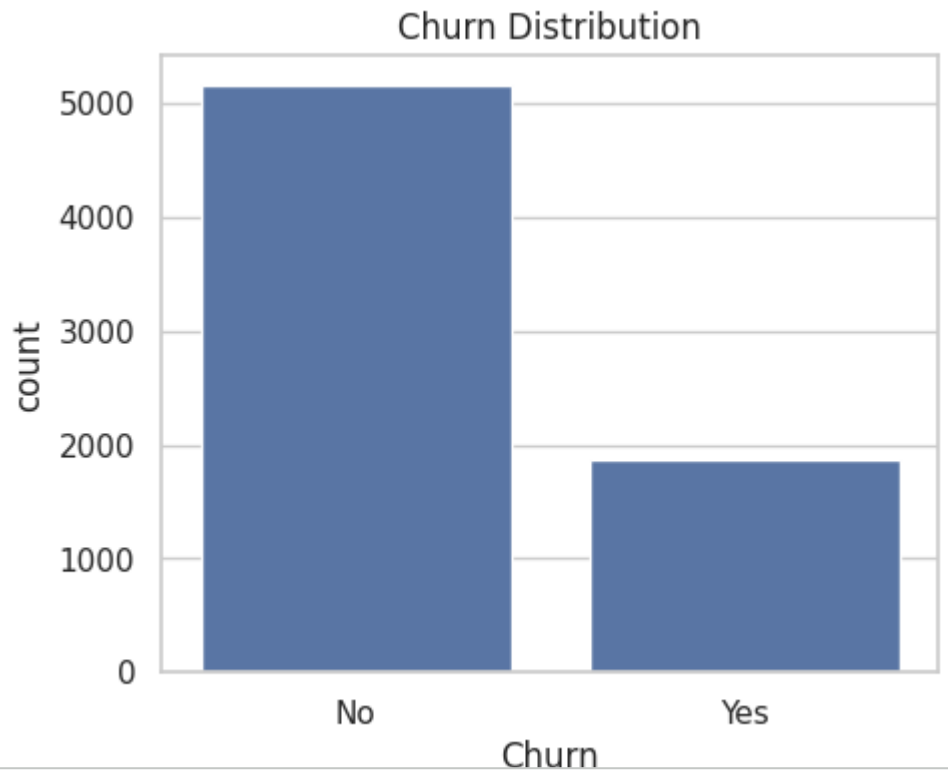
```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df.dropna(inplace=True)

if 'customerID' in df.columns:
    df.drop('customerID', axis=1, inplace=True)

df.head()
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	Onl:
0	Female	0	Yes	No	1	No	No phone service	DSL	No	
1	Male	0	No	No	34	Yes	No	DSL	Yes	
2	Male	0	No	No	2	Yes	No	DSL	Yes	
3	Male	0	No	No	45	No	No phone service	DSL	Yes	
4	Female	0	No	No	2	Yes	No	Fiber optic	No	

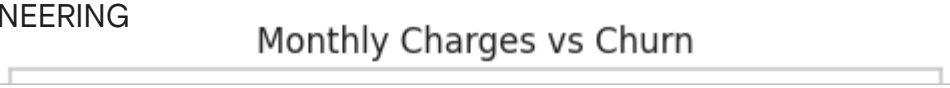
Next steps: Generate code with df New interactive sheet

ADVANCED EDA

```
plt.figure(figsize=(5,4))
sns.countplot(x='Churn', data=df)
plt.title("Churn Distribution")
plt.show()

plt.figure(figsize=(6,4))
sns.boxplot(x='Churn', y='MonthlyCharges', data=df)
plt.title("Monthly Charges vs Churn")
plt.show()

plt.figure(figsize=(6,4))
sns.boxplot(x='Churn', y='tenure', data=df)
plt.title("Tenure vs Churn")
plt.show()
```

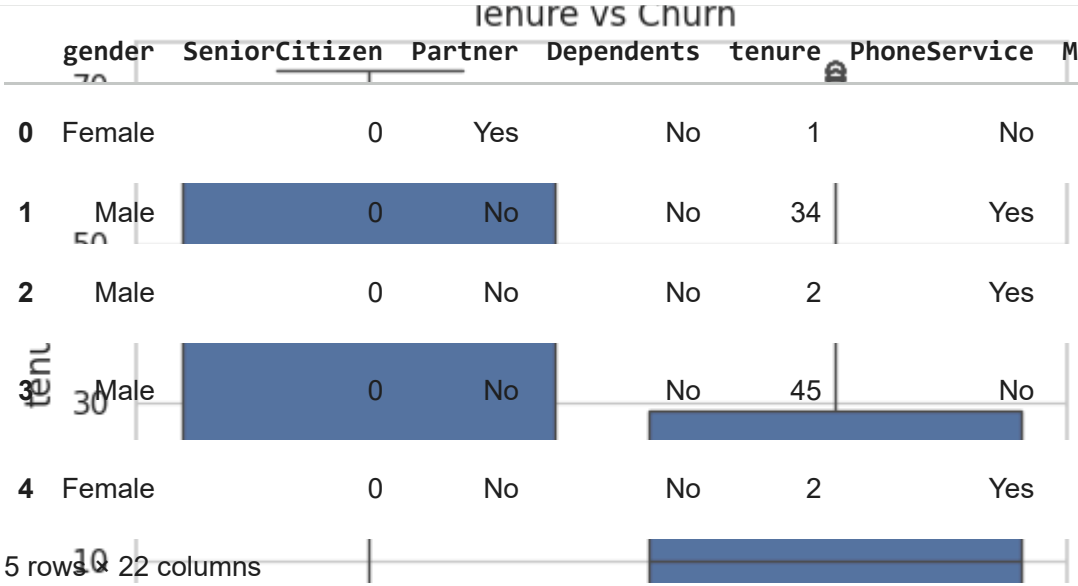


FEATURE ENGINEERING




```
df['tenure_group'] = pd.cut(
    df['tenure'],
    bins=[0,12,24,48,60,100],
    labels=['0-1yr','1-2yr','2-4yr','4-5yr','5+yr']
)

services = ['PhoneService','InternetService','OnlineSecurity',
            'OnlineBackup','DeviceProtection','TechSupport',
            'StreamingTV','StreamingMovies']

for col in services:
    if col in df.columns:
        df[col] = df[col].replace({'No internet service':'No',
```

```
                                    'No phone service':'No'})

df['num_services'] = (df[services] == 'Yes').sum(axis=1)

df.head()
```



	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	Onl
0	Female	0	Yes	No	1	No	No phone service	DSL	No	
1	Male	0	No	No	34	Yes	No	DSL	Yes	
2	Male	0	No	No	2	Yes	No	DSL	Yes	
3	Male	0	No	No	45	No	No phone service	DSL	Yes	
4	Female	0	No	No	2	Yes	No	Fiber optic	No	

5 rows × 22 columns

```
le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])

df.head()
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	Onli
0	0	0	1	0	1	0	1	0	0	
1	1	0	0	0	34	1	0	0	1	
2	1	0	0	0	2	1	0	0	1	
3	1	0	0	0	45	0	1	0	1	
4	0	0	0	0	2	1	0	1	0	

5 rows × 22 columns

MODEL BUILDING

```
import pandas as pd
if 'tenure_group' in df.columns and isinstance(df['tenure_group'].dtype, pd.CategoricalDtype):
    le_tenure = LabelEncoder()
    df['tenure_group'] = le_tenure.fit_transform(df['tenure_group'])

X = df.drop('Churn', axis=1)
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

MODEL EVALUATION

L.R.

```
lr = LogisticRegression(class_weight='balanced', max_iter=1000)
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
y_prob_lr = lr.predict_proba(X_test)[:,1]

print("Logistic Accuracy:", accuracy_score(y_test, y_pred_lr))
print("ROC-AUC:", roc_auc_score(y_test, y_prob_lr))
print(classification_report(y_test, y_pred_lr))
```

```
Logistic Accuracy: 0.738450604122246
ROC-AUC: 0.8299460063881225
              precision    recall  f1-score   support

           0       0.91      0.71      0.80      1033
           1       0.51      0.81      0.62       374

    accuracy                           0.74      1407
   macro avg       0.71      0.76      0.71      1407
weighted avg       0.80      0.74      0.75      1407
```

R.F.

```
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    class_weight='balanced',
    random_state=42
)

rf.fit(X_train, y_train)
```

```
y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:,1]

print("RF Accuracy:", accuracy_score(y_test, y_pred_rf))
print("RF ROC-AUC:", roc_auc_score(y_test, y_prob_rf))
print(classification_report(y_test, y_pred_rf))
```

```
RF Accuracy: 0.7604832977967306
RF ROC-AUC: 0.8305723944070279
              precision    recall  f1-score   support

           0       0.88      0.78      0.83      1033
           1       0.54      0.71      0.61       374

    accuracy                           0.76      1407
   macro avg       0.71      0.74      0.72      1407
weighted avg       0.79      0.76      0.77      1407
```

XGB

```
if xgb_available:
    xgb = XGBClassifier(
        eval_metric='logloss',
        use_label_encoder=False
    )

    xgb.fit(X_train, y_train)

    y_pred_xgb = xgb.predict(X_test)
    y_prob_xgb = xgb.predict_proba(X_test)[:,1]

    print("XGB Accuracy:", accuracy_score(y_test, y_pred_xgb))
    print("XGB ROC-AUC:", roc_auc_score(y_test, y_prob_xgb))
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [22:51:25] WARNING: /__w/xgboost/xgboost/s
Parameters: { "use_label_encoder" } are not used.
```

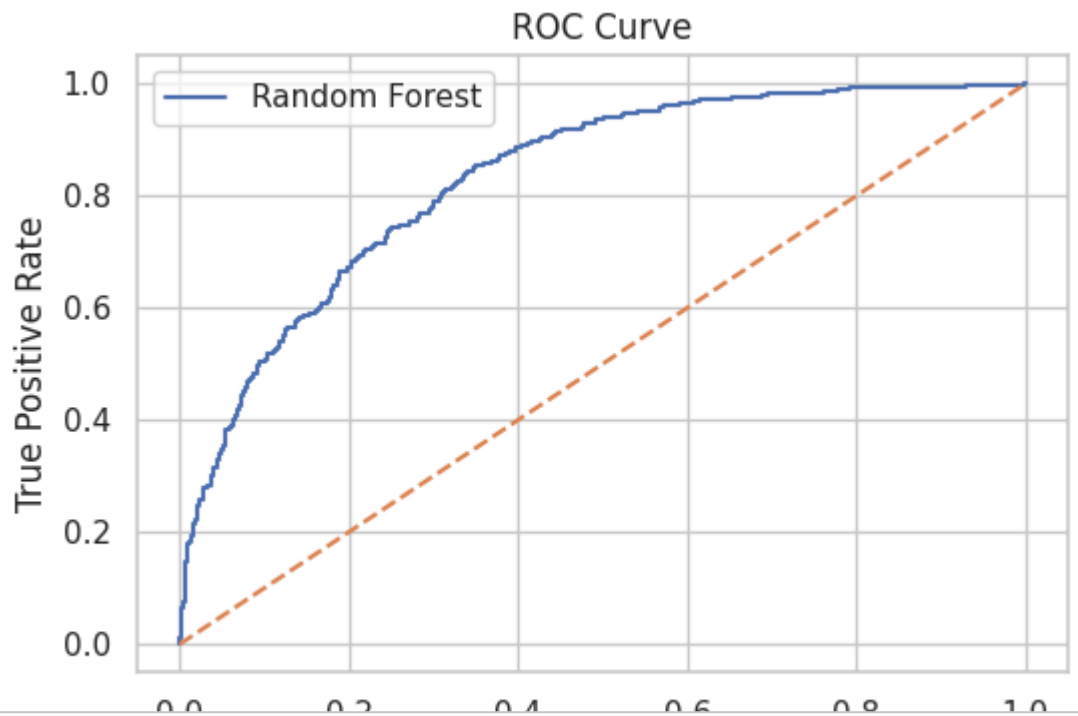
```
  bst.update(dtrain, iteration=i, fobj=obj)
XGB Accuracy: 0.7704335465529495
XGB ROC-AUC: 0.8073416817224117
```

```
models = {
    "Logistic": roc_auc_score(y_test, y_prob_lr),
    "RandomForest": roc_auc_score(y_test, y_prob_rf)
}

if xgb_available:
    models["XGBoost"] = roc_auc_score(y_test, y_prob_xgb)

pd.Series(models).sort_values(ascending=False)
```

	0
RandomForest	0.830572
Logistic	0.829946
XGBoost	0.807342

dtype: float64

```
fpr, tpr, _ = roc_curve(y_test, y_prob_rf)

plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label="Random Forest")
plt.plot([0,1],[0,1],'--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```




```
importances = rf.feature_importances_

feature_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

feature_df.head(10)
```

	Feature	Importance
14	Contract	0.187561
4	tenure	0.149383
17	MonthlyCharges	0.141461
18	TotalCharges	0.121290
7	InternetService	0.076009
19	tenure_group	0.057227
16	PaymentMethod	0.043886
20	num_services	0.031227
8	OnlineSecurity	0.022716
15	PaperlessBilling	0.021593

Next steps: Generate code with `feature_df` New interactive sheet

BUSINESS INSIGHTS

```
import shap

best_model = rf
explainer = shap.TreeExplainer(best_model)
shap_values = explainer.shap_values(X_train)

shap.summary_plot(shap_values, X_train)
```